

Tab 1

Product Pitch: The Syntax Scanner (Project "Math-Code")

The Vision

We are pivoting from a passive "Reading Game" to an active **"Math-Code Debugger."** The player isn't just walking through a site; they are an Inspector scanning "glitched" math symbols to restore the world's logic.

Business Viability: Cost and Risk

- **Minimal Dev-Hours (Cost):** Since the developer already has the walking mechanics and trigger zones, we don't need new "plumbing." We simply replace "Text Box A" with a "Scan Mechanic." This is a **low-cost pivot** because it reuses 90% of the existing code.
- **The Rubric Guardrail (Risk):** The biggest risk is submitting a game that is "too simple" for a college math course. By focusing on **Module 1: The Language of Math**, we align 100% with the academic requirements. We move from the risk of "just a game" to the certainty of a "Technical Academic Tool."

User Experience: Friction and Flow

- **Eliminating "Reading Fatigue" (Friction):** Currently, the user has to stop and read long text boxes. That is a **Friction Point** where players quit. In the "Scanner" model, the math *is* the gameplay. Finding the difference between a "Math Noun" (Expression) and a "Math Verb" (Sentence) becomes the "zap" moment.
- **The Discovery Loop (Flow):** We create a **High-Dopamine Flow** by making the environment react to the math. When a user correctly "zaps" an expression, a bridge completes or a door opens. The math isn't an obstacle to the game; the math *is* the key to the world.

Implementation Plan (The Sprint)

1. **Stage 1:** Use existing 2D triggers to identify "Corrupted Math" in the environment.
2. **Stage 2:** Player clicks/zaps the symbol.
3. **Stage 3:** A simple 2-choice UI appears: **[Is this an Expression?] or [Is this a Sentence?]**.
4. **Stage 4:** Correct choice = World restores. Incorrect choice = Penalty/Glitch.

We are pivoting from a passive "Reading Game" to an active **"Math-Code Debugger."** The player isn't just walking through a site; they are an Inspector scanning "glitched" math symbols to restore the world's logic.

Business Viability: Cost and Risk

- **Minimal Dev-Hours (Cost):** Since the developer already has the walking mechanics

and trigger zones, we don't need new "plumbing." We simply replace "Text Box A" with a "Scan Mechanic." This is a **low-cost pivot** because it reuses 90% of the existing code.

- **The Rubric Guardrail (Risk):** The biggest risk is submitting a game that is "too simple" for a college math course. By focusing on **Module 1: The Language of Math**, we align 100% with the academic requirements. We move from the risk of "just a game" to the certainty of a **"Technical Academic Tool."**

User Experience: Friction and Flow

- **Eliminating "Reading Fatigue" (Friction):** Currently, the user has to stop and read long text boxes. That is a **Friction Point** where players quit. In the "Scanner" model, the math *is* the gameplay. Finding the difference between a "Math Noun" (Expression) and a "Math Verb" (Sentence) becomes the "zap" moment.
- **The Discovery Loop (Flow):** We create a **High-Dopamine Flow** by making the environment react to the math. When a user correctly "zaps" an expression, a bridge completes or a door opens. The math isn't an obstacle to the game; the math *is* the key to the world.

Implementation Plan (The Sprint)

1. **Stage 1:** Use existing 2D triggers to identify "Corrupted Math" in the environment.
2. **Stage 2:** Player clicks/zaps the symbol.
3. **Stage 3:** A simple 2-choice UI appears: **[Is this an Expression?] or [Is this a Sentence?]**.
4. **Stage 4:** Correct choice = World restores. Incorrect choice = Penalty/Glitch.

For this pivot to work, we need to treat the **Syntax Scanner** as a lean product launch. Since you're operating as the **Technical Product Owner**, the goal is to align these roles with the **Sprint Stages** to ensure we hit the academic rubric without bloating the dev-hours.

-----Project "Math-Code" Resource Allocation

1. Ranzel Virtucio | Technical Product Owner & Logic Lead

- **Focus: The Rubric Guardrail (Risk Management).**
- **Responsibilities:**
 - Oversee the **Implementation Plan** (Stages 1–4) to ensure 100% alignment with **Module 1: The Language of Math**.
 - Define the "Corrupted Math" dataset: selecting the specific **Expressions** (Nouns) and **Sentences** (Verbs) that provide the "zap" moment for users.
 - Finalize the **Business Viability** documentation, ensuring the project is framed as a "Technical Academic Tool" rather than just a simple game.

2. Josiah Jaravilla Facultad | Systems Architect (The "Plumbing" Specialist)

- **Focus: Minimal Dev-Hours (Cost Efficiency).**
- **Responsibilities:**
 - Manage the **Stage 1 & 2** implementation: repurposing the existing 2D triggers and walking mechanics into the **Scan Mechanic**.
 - Ensure the system reuses **90% of the existing code** to keep production costs low.
 - Handle the logic for the "World Restoration" or "Penalty/Glitch" triggers based on user input.

3. Kalani Phoenix Guinto | UX Designer (Flow & Feedback)

- **Focus: Eliminating Friction & High-Dopamine Flow.**
- **Responsibilities:**
 - Design the **Stage 3 UI**: Creating the 2-choice interface that allows players to "zap" symbols without **Reading Fatigue**.
 - Develop the visual "Glitch" and "Restore" assets: ensuring the environment reacts instantly when a user solves a math problem.
 - Audit the **Discovery Loop** to make sure the math feels like a "key" rather than an obstacle.

4. John Gabriel Tolentino | Quality Assurance (QA) & Integration

- **Focus: The Technical Guardrail.**
- **Responsibilities:**
 - Test the **Trigger Zones** to ensure the "Inspector" persona feels responsive and logical.
 - Verify that all **Module 1** mathematical definitions used in the game are academically accurate.
 - Monitor the transition from "Text Box A" to the "Scan Mechanic" to prevent any breaking bugs during the pivot.

Corrupted Math Dataset to use

Raw implementation for the corrupted math dataset
expressions or math nouns

1. $3x + 5$
2. $x^2 + 4$
3. $a + b$
4. 14
5. square root of 9

sentences or math verbs

1. $3x + 5 = 11$
2. $2 < 4$
3. $x = y$
4. $1 + 1 = 2$
5. $10 > 20$

Business viability cost and risk

defining this dataset right now minimizes our dev hours because the team does not have to spend time researching what math to include. it mitigates risk by locking our mechanics directly to the module 1 syllabus so we pass the rubric without any scope creep.

user experience friction and flow

keeping these symbols concise eliminates reading fatigue. the cognitive difference between scanning a polynomial noun and an equation verb creates that instant zap moment. this maintains our high dopamine flow.

The system collapses without this dataset because the trigger zones will have no functional parameters which turns the project back into a passive reading simulator instead of an active logic scanner.

Plan

Implementation Plan: Module 1 Alignment (Stages 1–4)

1. **Stage 1: Data Structuring (The Spec)**
 - Lock the finalized "Corrupted Math Dataset" (the 5 Nouns and 5 Verbs) into the core logic.
 - This ensures the environment only generates obstacles strictly defined by the Module 1 syllabus, guaranteeing 100% academic compliance before any interface is drawn.
2. **Stage 2: Engine Repurposing (The Plumbing)**
 - Bind the existing 2D trigger zones to the dataset.
 - When the player intersects with a zone, the engine simply calls an item from the locked array rather than generating new physics or text assets.
3. **Stage 3: Interface Layer (The Flow)**
 - Deploy the binary decision UI for the "Scan Mechanic."
 - Presenting only two interactive buttons ("Expression" or "Sentence") upon trigger activation enforces immediate user action over passive consumption.
4. **Stage 4: Compliance Telemetry (The Guardrail)**
 - Execute the final QA sweep against the academic rubric.
 - Every "zap" interaction must trace directly back to the definitions of mathematical nouns and verbs to certify the system as a valid evaluation method.

-----Business Viability Documentation: The Technical Academic Tool

- **Business Viability (Cost/Risk):**
 - Framing the application strictly as a "**Technical Academic Tool**" entirely mitigates the risk of failing the grading rubric by anchoring every mechanic to the Module 1 syllabus.
 - By reusing 90% of the existing movement logic to drive the new scanning feature, we keep development costs exceptionally low and avoid the technical debt associated with building a complex, generic game engine.
- **User Experience (Friction/Flow):**
 - Replacing traditional, text-heavy tutorial boxes with an immediate, binary logic choice completely eliminates **reading fatigue**.
 - By making the math itself the primary mechanism for world restoration, the player stays locked in a **high-dopamine debugging loop**, transforming abstract algebraic concepts into tangible, frictionless interactions.
- **System Risk:** The system collapses without this formalized roadmap because ambiguous project framing will inevitably cause the team to abandon the Spec-First architecture, resulting in fatal scope creep and a failed academic submission.

April 27, 2026 | 10:45 AM PST